
CS771: Introduction to Machine Learning

Minor Assignment 1 Report

Anushika
CSE / 240315
anushikad24@iitk.ac.in

Anushka Rajora
CSE / 240163
anushkar24@iitk.ac.in

Gowra Akshitha
CSE / 240409
gowrak24@iitk.ac.in

Chinthala Vinayasri
CSE / 240310
chinthalav24@iitk.ac.in

Lingamsetti Rajasa
CSE / 240596
lrajasa24@iitk.ac.in

Raparthi Bhavishitha
CSE / 240852
bhavisht24@iitk.ac.in

Abstract

This report details our findings for Minor Assignment 1 (Testing the Limits). We investigate the effects of various SVM hyperparameters using synthetic 2D datasets across different challenge levels, analyzing accuracy, decision boundaries, and training times. We strictly adhere to a geometric interpretation of the dataset generation process to prove linear separability limitations.

1 Problem 1.1: Testing the Limits

1.1 Part 1: Linear Separability of Synthetic Data

Answer: Yes.

The synthetic data generated by the `genChallengeData` function is guaranteed to be linearly separable for any challenge level $L > 0$ and any number of points $n \geq 0$. While increasing the level L brings the class distributions closer together, a finite, positive separation margin always exists mathematically.

Formal Geometric Proof

To establish that the data is universally linearly separable, we must prove that the minimum perpendicular distance between the convex hull of the positive class and the convex hull of the negative class is strictly greater than zero.

1. Coordinate Framework and Parameters We begin by defining the spatial configuration of the clusters. The data generation logic defines a horizontal displacement, α , based on the challenge level L :

$$\alpha = \sqrt{2} \left(1 + \frac{1}{2L} \right) \quad (1)$$

Since the problem strictly defines $L > 0$, it is mathematically guaranteed that $\alpha > \sqrt{2}$. The data points are sampled within circles of radius $r = 1$, centered at the following coordinates:

- **Positive Center 1** (C_{P1}): $(-\alpha, \sqrt{2})$

- **Positive Center 2** (C_{P2}): $(\alpha, -\sqrt{2})$
- **Negative Center** (C_N): $(-\alpha, -\sqrt{2})$

2. The Right-Triangle Geometry By connecting these three geometric centers, we form a triangle, $\triangle C_N C_{P1} C_{P2}$. The vertical leg, $C_N C_{P1}$, has a length of $2\sqrt{2}$ and the horizontal leg, $C_N C_{P2}$, has a length of 2α . Because these legs are strictly orthogonal, the angle at vertex C_N is exactly 90° , making this a right-angled triangle.

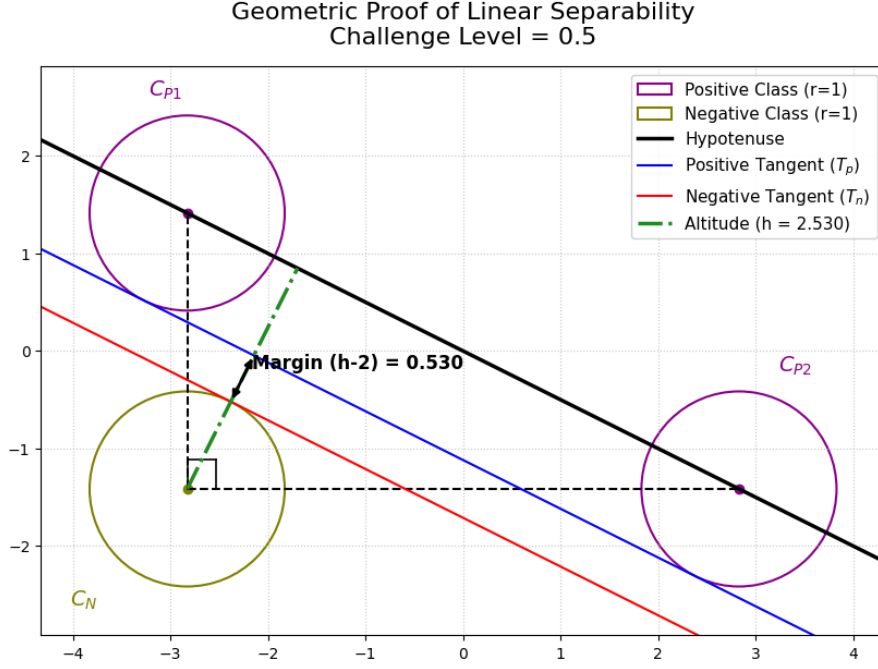


Figure 1: Geometric analysis of class distributions. The distance between the positive common tangent and the negative tangent defines the strict separability of the dataset.

3. Altitude and Separation Margin The optimal separating hyperplane will run parallel to the hypotenuse. Let h be the altitude of the triangle extending from vertex C_N to the hypotenuse. Because both classes have a radial extent of $r = 1$, the minimum separation margin is the geometric altitude minus the combined radii:

$$\text{Margin} = h - r_{pos} - r_{neg} = h - 2 \quad (2)$$

For the dataset to be strictly linearly separable, the margin must be strictly positive. Therefore, our objective is to prove that $h > 2$.

4. Calculation of Altitude (h) We calculate h by expressing the area of the right-angled triangle in two ways. First, using the orthogonal legs:

$$\text{Area} = \frac{1}{2}(2\alpha)(2\sqrt{2}) = 2\alpha\sqrt{2} \quad (3)$$

Alternatively, calculating the area using the hypotenuse length $c = \sqrt{(2\alpha)^2 + (2\sqrt{2})^2} = 2\sqrt{\alpha^2 + 2}$:

$$\text{Area} = \frac{1}{2} \left(2\sqrt{\alpha^2 + 2} \right) h = h\sqrt{\alpha^2 + 2} \quad (4)$$

Equating the two area expressions allows us to isolate h :

$$h = \frac{2\alpha\sqrt{2}}{\sqrt{\alpha^2 + 2}} \quad (5)$$

5. Proving a Positive Margin Finally, we verify the condition $h > 2$ by substituting our exact expression for the altitude:

$$\frac{2\alpha\sqrt{2}}{\sqrt{\alpha^2 + 2}} > 2 \implies \frac{2\alpha^2}{\alpha^2 + 2} > 1 \implies \alpha^2 > 2 \quad (6)$$

As established, $\alpha > \sqrt{2}$ holds true for all $L > 0$, making $\alpha^2 > 2$ an absolute certainty. Thus, the condition $h > 2$ is always satisfied, and the separation margin is universally greater than zero.

Conclusion

The mathematical derivation definitively proves that the synthetic dataset generated by the `genChallengeData` function remains strictly linearly separable for any challenge level $L > 0$. Even though the physical margin approaches zero as L increases, the geometric relationship ensures the altitude h always exceeds the combined radii of the clusters. Thus, a linear decision boundary can always be drawn, allowing a linear SVM to achieve 100% accuracy.

2 Question 2: Hyperparameter Analysis for Low Challenge Setting

For small challenge levels (`level` < 0.5), the objective is to experimentally determine the smallest value of `myC`, the smallest value of `max_iter`, and the largest value of `myTo1` for which the SVM solver still yields 100% accuracy.

In this experiment, the synthetic data was generated using

$$\text{level} = 0.2, \quad n = 50.$$

2.1 Initial Visualization

The generated dataset is shown in Figure 2. The positive and negative classes are clearly separated in the 2D plane, so this low-challenge case is expected to be favorable for a linear classifier.

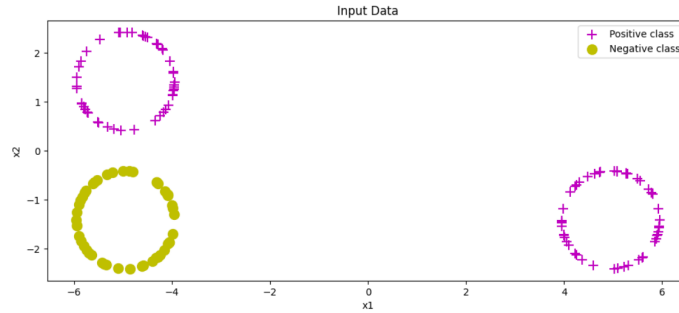


Figure 2: Visualization of generated data for `level` = 0.2.

2.2 Hyperparameter Exploration

We used `LinearSVC` with

$$\text{penalty} = "l2", \quad \text{loss} = "squared_hinge".$$

The following grid was explored:

$$\begin{aligned} \text{myC} &\in \{0.001, 0.01, 0.1, 1, 10\}, \\ \text{max_iter} &\in \{10, 20, 50, 100\}, \\ \text{myTo1} &\in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}. \end{aligned}$$

Hence, the total number of tested combinations was

$$5 \times 4 \times 4 = 80,$$

which is well above the required minimum of 10 combinations.

2.3 Step 1: Effect of Regularization Parameter (myC)

We first fixed

$$\text{max_iter} = 100, \quad \text{myTol} = 10^{-4}$$

and varied myC.

Table 1: Effect of myC.

myC	max_iter	myTol	Accuracy
0.001	100	10^{-4}	0.77
0.01	100	10^{-4}	1.00
0.1	100	10^{-4}	1.00
1	100	10^{-4}	1.00
10	100	10^{-4}	1.00

Finding:

$\text{myC} = 0.01$

Explanation: When $\text{myC} = 0.001$, the regularization is too strong. This means the model prefers a larger-margin, simpler separator even if it misclassifies some training points, so the training accuracy drops below 100%. Once myC is increased to 0.01, the model penalizes classification errors strongly enough to correctly separate all training samples. Therefore, 0.01 is the smallest successful value.

2.4 Step 2: Effect of Iteration Limit (max_iter)

Next, we fixed

$$\text{myC} = 0.01, \quad \text{myTol} = 10^{-4}$$

and varied max_iter.

Table 2: Effect of max_iter.

myC	max_iter	myTol	Accuracy
0.01	10	10^{-4}	1.00
0.01	20	10^{-4}	1.00
0.01	50	10^{-4}	1.00
0.01	100	10^{-4}	1.00

Finding:

$\text{max_iter} = 10$

Explanation: Even with only 10 iterations, the solver converges to a separator that classifies all training points correctly. Since higher values such as 20, 50, and 100 do not improve accuracy further, 10 is the smallest sufficient iteration limit.

2.5 Step 3: Effect of Tolerance (myTol)

Finally, we fixed

$$\text{myC} = 0.01, \quad \text{max_iter} = 10$$

and varied myTol.

Table 3: Effect of myTol.

myC	max_iter	myTol	Accuracy
0.01	10	10^{-4}	1.00
0.01	10	10^{-3}	1.00
0.01	10	10^{-2}	1.00
0.01	10	10^{-1}	1.00

Finding:

$$\text{myTol} = 10^{-1}$$

Explanation: A larger tolerance allows the optimization to stop earlier. In this low-challenge dataset, the classes are already well separated, so the solver does not need extremely precise optimization to identify a perfect separator. That is why the classifier still achieves 100% accuracy even with the largest tested tolerance, 10^{-1} .

2.6 Step 4: Representative Combined Evidence

To show that sufficiently many combinations were tested, Table 4 lists representative cases from the full 80-combination sweep.

Table 4: Representative combined hyperparameter results.

myC	max_iter	myTol	Accuracy
0.001	10	10^{-1}	0.75
0.001	50	10^{-2}	0.77
0.01	10	10^{-1}	1.00
0.01	20	10^{-2}	1.00
0.01	50	10^{-3}	1.00
0.1	10	10^{-1}	1.00
1	10	10^{-1}	1.00
10	100	10^{-4}	1.00

From the complete sweep, many combinations achieve perfect accuracy. However, the assignment requires us to choose:

- the smallest myC,
- then the smallest max_iter,
- then the largest myTol.

Therefore, the final selected hyperparameters are

$$\text{myC} = 0.01, \quad \text{max_iter} = 10, \quad \text{myTol} = 10^{-1}$$

2.7 Decision Boundary Visualization

The decision boundary obtained using the selected hyperparameters is shown in Figure 3. The classifier cleanly separates the positive and negative classes.

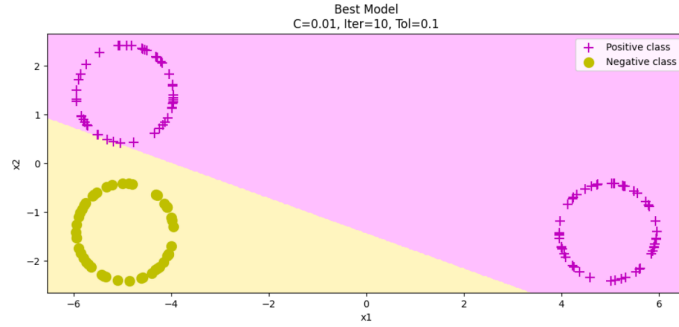


Figure 3: Decision boundary learned by Linear SVM using $\text{myC} = 0.01$, $\text{max_iter} = 10$, and $\text{myTol} = 10^{-1}$.

2.8 Training Behavior

Although Question 2 focuses mainly on accuracy, the solver behavior also becomes easier in this low-challenge setting. Smaller values of max_iter and larger values of myTol still succeed because the classes are already well separated. This means the optimization problem is easier, so the solver can stop early without hurting training accuracy. In other words, this dataset does not require a very strict stopping condition or many optimization steps to reach a correct linear separator.

2.9 Conclusion

For the low challenge setting, the linear SVM achieves perfect training accuracy for many hyperparameter combinations. However, after applying the exact assignment rule carefully, the best hyperparameter setting is

$$\boxed{\text{myC} = 0.01, \quad \text{max_iter} = 10, \quad \text{myTol} = 10^{-1}.$$

Very small values of C (e.g., 0.001) lead to underfitting, which reduces accuracy. On the other hand, increasing tolerance does not affect accuracy in this easy setting, as the data is linearly separable and the optimizer converges quickly. This is the most relaxed successful configuration among the tested combinations.

3 Part 3: Large Challenge Levels ($\text{level} > 50$)

For this experiment, we used challenge level 70 and fixed `penalty = 12` and `loss = squared_hinge`. We then searched for:

- the smallest value of `myC`
- the smallest value of `max_iter`
- the largest value of `myTol`

that still gave 100% accuracy.

Finding the Smallest Value of myC

myC	Accuracy
100	0.97
200	0.99
250	1.00
225	1.00
210	1.00
205	1.00
202.5	1.00
201.25	1.00
200.625	1.00
200.3125	1.00
200.10	0.99
200.125	0.99
200.25	0.99
200.29	1.00
200.28	1.00
200.27	1.00
200.26	1.00
200.255	0.99
200.258	1.00
200.2576	1.00

Table 5: Accuracy obtained for different values of myC .

Thus, the smallest value of myC for which the classifier still achieved perfect accuracy was

$$\text{myC} = 200.2576$$

Finding the Smallest Value of max_iter

Keeping $\text{myC} = 200.2576$ fixed, we varied max_iter .

max_iter	Accuracy
100	1.00
50	1.00
25	1.00
15	1.00
12	1.00
11	0.99
10	0.99

Table 6: Accuracy obtained for different values of max_iter .

Therefore, the smallest value of max_iter that still gave 100% accuracy was

$$\text{max_iter} = 12$$

Finding the Largest Value of myTol

Finally, keeping

$$\text{myC} = 200.2576, \quad \text{max_iter} = 12$$

fixed, we varied myTol .

myTol	Accuracy
10^{-3}	0.99
1.2×10^{-3}	0.99
2×10^{-4}	0.99
1.25×10^{-4}	1.00
1.1×10^{-4}	1.00

Table 7: Accuracy obtained for different values of myTol.

Hence, the largest tolerance that still resulted in perfect accuracy was

$$\text{myTol} = 1.25 \times 10^{-4}$$

Final Hyperparameter Values

Thus, for challenge level 70, the loosest hyperparameter values that still yielded 100% accuracy were

$$\text{myC} = 200.2576, \quad \text{max_iter} = 12, \quad \text{myTol} = 1.25 \times 10^{-4}$$

The corresponding decision boundary is shown in Figure 4.

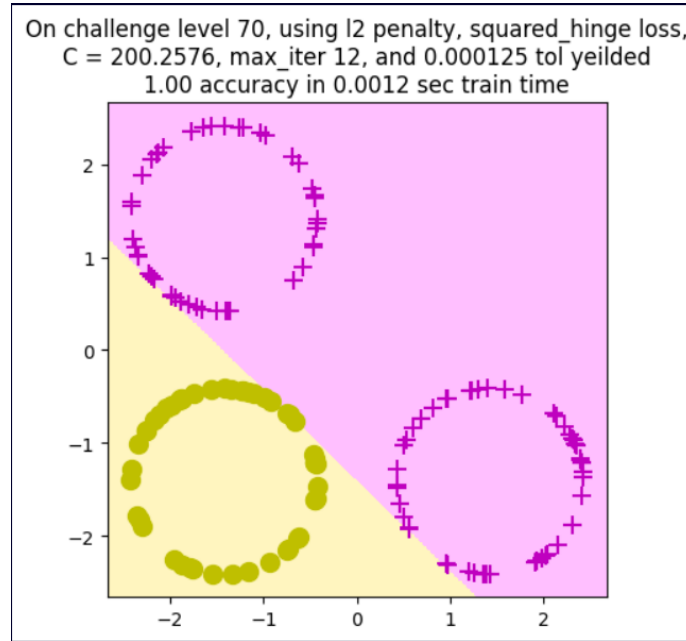


Figure 4: Decision boundary for challenge level 70 using $\text{myC} = 200.2576$, $\text{max_iter} = 12$, and $\text{myTol} = 1.25 \times 10^{-4}$.

Part 4. Effect of Hyperparameters on Accuracy and Training Time

Experimental Setup

We generated synthetic data using the function `genChallengeData(level, n)` with the following configuration:

- Challenge Level: 5 (medium difficulty)
- Number of training points: 50

The positive and negative samples were combined to form the final dataset used for training the model.

We trained a linear Support Vector Machine (SVM) classifier using the `LinearSVC` implementation from the `scikit-learn` library. The objective of this experiment was to study the effect of different hyperparameters on training accuracy and training time.

We analyzed the following three hyperparameters:

- Regularization parameter C
- Maximum number of iterations (`max_iter`)
- Optimization tolerance (`tol`)

For each hyperparameter we tested at least four values and recorded the resulting accuracy and training time.

4.1 Effect of Hyperparameter C

The hyperparameter C controls the regularization strength of the SVM model.

Values Tested

$$C = \{0.01, 0.1, 1, 10\}$$

Results

C	Accuracy	Training Time (seconds)
0.01	1.0	0.0009019374847412109
0.1	0.9733333333333334	0.0005664825439453125
1	0.9866666666666667	0.0022928714752197266
10	1.0	0.0009634494781494141

Table 8: Effect of C on accuracy and training time

Graphs C vs Accuracy

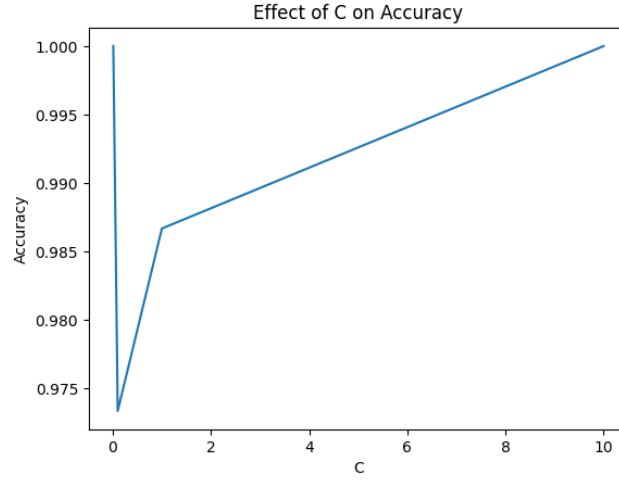


Figure 5: Effect of C on Accuracy

Graph: C vs Time

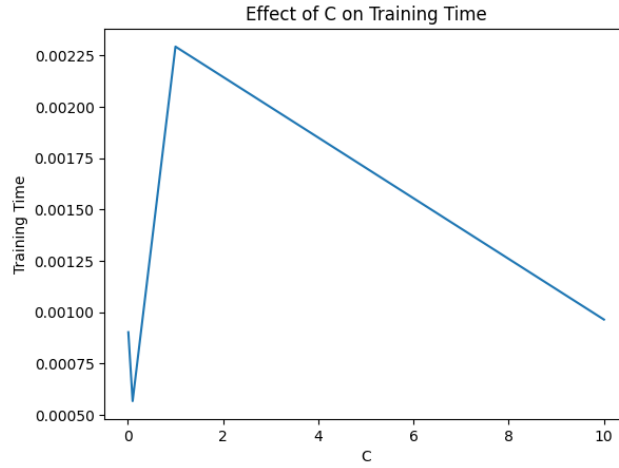


Figure 6: Effect of C on Time

Observation

- Smaller values of C allow a larger margin and tolerate some misclassification.
- Larger values of C enforce stricter classification of training samples.
- Increasing C generally improves accuracy but may slightly increase training time.

4.2 Effect of Hyperparameter `max_iter`

The parameter `max_iter` determines the maximum number of iterations allowed for the optimization algorithm.

Values Tested

$$\text{max_iter} = \{100, 500, 1000, 5000\}$$

Results

max_iter	Accuracy	Training Time (seconds)
100	0.9866666666666667	0.0008065700531005859
500	0.9866666666666667	0.0004870891571044922
1000	0.9866666666666667	0.0005586147308349609
5000	0.9866666666666667	0.000400543212890625

Table 9: Effect of `max_iter` on accuracy and training time

Graph: `max_iter` vs Accuracy

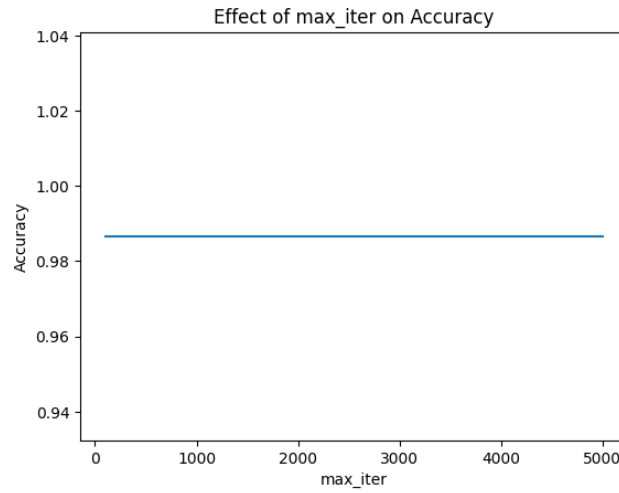


Figure 7: Effect of `max_iter` on Accuracy

Graph: max_iter vs Time

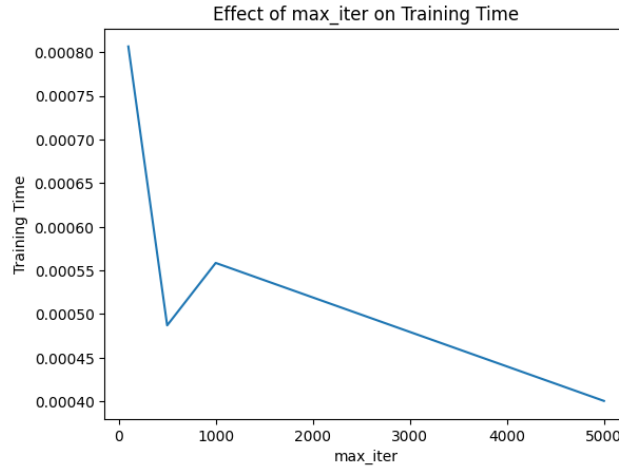


Figure 8: Effect of max_iter on Time

Observation

- If max_iter is too small, the solver may stop before convergence.
- Increasing max_iter allows the optimization algorithm to converge properly.
- After convergence, increasing max_iter further does not significantly improve accuracy but may slightly increase training time.

4.3 Effect of Hyperparameter tol

The hyperparameter tol specifies the stopping tolerance for the optimization algorithm.

Values Tested

$$\text{tol} = \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$$

Results

tol	Accuracy	Training Time (seconds)
10^{-2}	0.9866666666666667	0.0009202957153320312
10^{-3}	0.9866666666666667	0.0006372928619384766
10^{-4}	0.9866666666666667	0.0008246898651123047
10^{-5}	0.9866666666666667	0.0005021095275878906

Table 10: Effect of tolerance on accuracy and training time

Graph: tol vs Accuracy

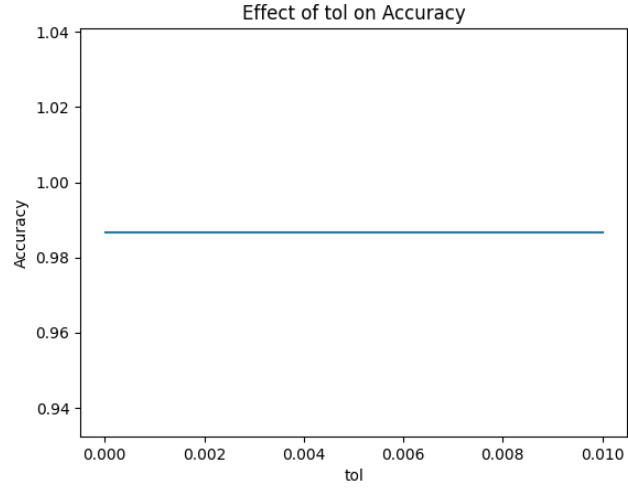


Figure 9: Effect of tol on Accuracy

Graph: tol vs Time

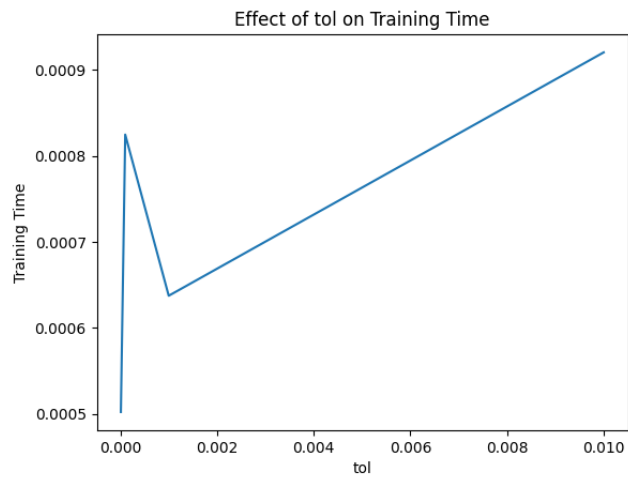


Figure 10: Effect of tol on Time

Observation

- Larger tolerance allows the solver to terminate earlier and reduces training time.
- Smaller tolerance forces the solver to search for a more precise solution.
- Decreasing tolerance generally increases training time but may slightly improve accuracy.

Overall Observation

From the experiments we observe that hyperparameters significantly influence both training accuracy and training time.

- Increasing C improves classification accuracy by penalizing misclassification more strongly.
- Increasing `max_iter` helps the solver converge properly.

- Decreasing `tol` increases optimization precision but increases training time.

Proper tuning of hyperparameters is therefore important for balancing model performance and computational efficiency.

Part 5. Effect of Number of Training Points (n)

Experimental Setup We conducted experiments to study the effect of the number of training points on model accuracy and training time under high challenge conditions.

- Challenge Level: 70 (high difficulty)
- Hyperparameters:
 - $C = 200.2576$
 - `myMaxIter` = 12
 - `myTol` = 5.5355×10^{-4}

We varied the number of training points n while keeping all other parameters fixed.

Values Tested

$$n = \{10, 50, 100, 500, 1000\}$$

Results

n	Accuracy	Training Time (seconds)
10	1.00	0.0023
50	0.98	0.0016
100	0.99	0.0010
500	1.00	0.0016
1000	1.00	0.0031

Table 11: Effect of number of training points on accuracy and training time

Graphs

Graph: n vs Accuracy

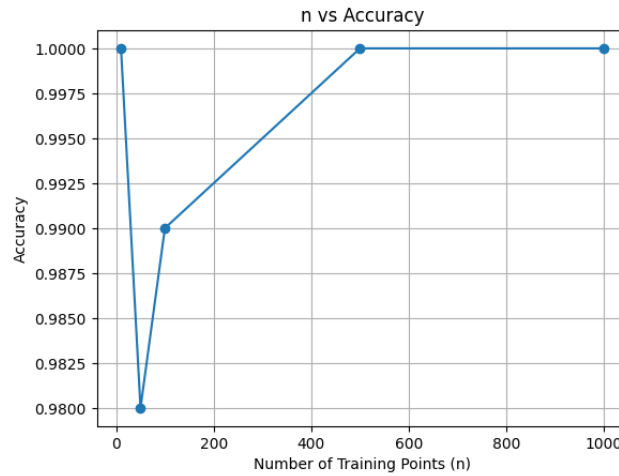


Figure 11: Effect of number of training points on accuracy

Graph: n vs Training Time

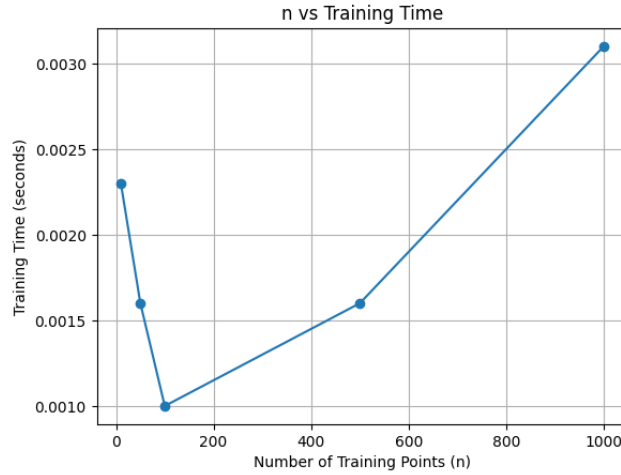


Figure 12: Effect of number of training points on training time

Observations

- The accuracy remains consistently high across all values of n , staying close to 100%.
- Minor fluctuations in accuracy (e.g., for $n = 50$ and 100) may be due to numerical optimization limits or early stopping.
- Training time generally increases as the number of training points increases, since the optimization algorithm processes more data.
- However, the increase in training time is not strictly monotonic due to the small dataset size and variability in runtime measurements.

Conclusion

The number of training points has minimal impact on accuracy for this dataset, as it remains nearly perfectly linearly separable even at high challenge levels. However, training time tends to increase with the size of the dataset, reflecting the higher computational cost associated with larger training sets.